

**1. [10 marks] Listing 1 describes a convolutional neural network used for classification. Considering the convolution and dense layers only, calculate the number of multiplications required to perform one classification operation, that is, classify a single input image. Show your work**

| Layer (type)              | Output Shape      | Param # |
|---------------------------|-------------------|---------|
| input_layer (InputLayer)  | (None, 28, 28, 1) | 0       |
| conv2d_0 (Conv2D)         | (None, 26, 26, 1) | 10      |
| max_pool_0 (MaxPooling2D) | (None, 13, 13, 1) | 0       |
| flatten (Flatten)         | (None, 169)       | 0       |
| dense_0 (Dense)           | (None, 100)       | 17,000  |
| dropout (Dropout)         | (None, 100)       | 0       |
| dense_1 (Dense)           | (None, 10)        | 1,010   |

Convolution works by sliding a filter/kernel of a given size over the input image. Here the kernel size is 3x3, meaning that the kernel aligns with 3x3 patches of pixels. Convolution is then applied between them by doing element wise multiplication and then summing the products into a single number. This means that for each “slide” of the kernel there are 9 multiplications. Furthermore since we only have 1 filter this means that there is only 1 kernel and 1 output channel.

We can see that the output shape of the input layer is 28x28. However, the output shape of the conv2d is 26x26. This means that the input image was not padded. To ensure that each filter value is multiplied against an existing image pixel value, the filter must not be applied to the outer border of pixels. If it was then the outer edge of filter values would not have a set of pixel values to be multiplied against. Hence, to ensure only valid convolution operations the image must be cut down to  $(28 - 1 - 1) \times (28 - 1 - 1) = 26 \times 26$ .

Overall, this means that there are  $26 \times 26 \times 9 = 6,084$  multiplications for the 2d convolution

On the other hand the 0th dense layer is doing matrix multiplication. The output of the flatten layer is (1,169) and the dense layer is initialized to have 100 neurons. This means that there is a  $(1,169) \times (169,100)$  matrix multiplication resulting in a shape of (1, 100). This is proven by the

output shape of the dense layer in the image above. Hence, the number of multiplications from this matrix operation is  $169 \times 100 = 16900$ .

For the second dense layer, the logic remains the same. The input is  $(1,100)$  multiplied by a weight matrix of size  $(100,10)$ . This results in  $100 \times 10 = 1000$  multiplications.

Therefore, the total number of multiplications between the convolutional and dense layers is  $6,084 + 19,900 + 1000 = 23,984$

**2. [10 marks] The network is to perform 2E6 classifications/second. Assuming a clock frequency of 100 MHz, how many DSP48s are required to implement the network in an FPGA. Show your working. You need only consider the DSP48's required to implement the convolutional and dense layers.**

If 1 classification takes 23,984 multiplications, this means the required multiplication rate is

$$= 2\text{e}6 \text{ classifications / sec} \times 23,984 \text{ multiplications / 1 classification} = 47,968\text{e}6 \text{ mult / sec}$$

If the clk frequency is 100 MHz this means that every 100MHz a DSP can do a multiplication operation. This corresponds to a period of 10 ns. To get the number of DSPs =

$$10 \text{ ns / 1 mult} \times 47,968\text{e}6 \text{ mult / 1 sec} = 479.68 = 480 \text{ DSPs}$$

We need at least 480 DSPs to implement the convolutional and dense layers.

## Lab Report:

### 1) Python Script:

The first phase of this final lab is to extract the kernel and bias terms from the convolutional layer.

```
x = Conv2D(1, (3, 3), activation='relu', kernel_initializer='he_uniform',  
name='conv2d_0')(input_layer)
```

From the keras tensorflow implementation in python, we can see that there is only 1 kernel of size 3x3. This means that we should expect 9 kernel values and 1 bias term. The following equation below demonstrates how convolution is similar to an inner product calculation. However now at the end we also then add the bias term.

$$\omega * F(x, y) = \left( \sum_{\delta x = -k_i}^{k_i} \sum_{\delta y = -k_j}^{k_j} \omega(\delta x, \delta y) \cdot F(x + \delta x, y + \delta y) \right) + \omega_{bias}$$

where  $\omega_{bias} \in \mathbb{R}$  is the bias of the kernel  $\omega$

It is important to note that the convolution requires a 3x3 grid of pixels but outputs only 1 value. This value is placed in the output at the center pixel/kernel position that is used for calculations.

The activation function is set to Relu which simply means that all inputs  $< 0$  are set to 0. If the input is  $\geq 0$  then the input = output. This means that once the convolution operation is applied the Relu function is applied to this value.

The next step is to extract the kernel values and bias term and write them to a .h file:

```
def write_conv_weights_h(model, filename="cnn_conv_weights.h"):  
    W, b, kernel_flat = get_conv2d0_params(model)  
  
    with open(filename, "w") as f:  
        f.write('#include "cnn_conv.h"\n\n')  
  
        f.write("// conv2d_0: flattened 3x3 kernel -> FILTER_SIZE=9\n")  
        f.write("static const weight_t conv2d_0_w[FILTER_SIZE] = {\n  ")  
        f.write(", ".join(_fmt(v) for v in kernel_flat))  
        f.write("\n};\n\n")  
  
        f.write("static const weight_t conv2d_0_b = ")  
        f.write(_fmt(b[0]))
```

```
f.write("\n")

print(f"Saved weights header to {filename}")
```

This results in the following .h file in the hls implementation:

```
#include "cnn_conv.h"

// conv2d_0: flattened 3x3 kernel -> FILTER_SIZE=9
static const weight_t conv2d_0_w[FILTER_SIZE] = {
    6.0848844051e-01, 1.0053322315e+00, -2.9141962528e-01, 5.1448982954e-01, 7.1830844879e-01,
    8.5052061081e-01, 3.1382909417e-01, 2.5166204572e-01, 6.9357149303e-02
};

static const weight_t conv2d_0_b = -2.0171105862e-03;
```

We can correctly see how the kernel/filter size correctly consists of 9 elements and only 1 bias term.

## 2) Assigning the number of Integer bits:

The next task in the python script is to export the input and output for the conv2D into 2 separate .dat files. From the model description in the previous section, we can see how the conv2D directly takes the 28x28 input image. Therefore the input should be a 28x28 image formatted into a .dat file of a flattened image of size 784. The output of this conv2D, consists of the convolution output with Relu applied. As mentioned previously, as there is no padding applied the output is of size 26x26. To make the testbench more flexible, the convolution values before and after the Relu function are both written to the .dat. This means that the .dat file consists of two flattened arrays each of size  $26 \times 26 = 676$ .

The final step in the python notebook before implementing this into HLS is to understand the range of all the values used. As we must use fixed point representations, the number of integer bits must be specified in order to maximize the amount of precision and thus accuracy in HLS. To to this, I implemented another function in my python script which prints the max and min of the various values passed to HLS:

```
===== CONV WEIGHT RANGES =====
conv2d_0_w min: -0.291420, max: 1.005332
conv2d_0_b min: -0.002017, max: -0.002017

===== INPUT RANGES =====
X min: 0.000000, max: 1.000000
```

```
===== CONV ACTIVATION RANGES =====
conv z min: -0.293437, max: 4.329971
conv relu min: 0.000000, max: 4.329971
=====
```

These results show that the range of the kernel 3x3 values has a max of 1.005. This corresponds to a range of  $[-2, 1.99]$  and thus a range of 1 bit ( $2^1$ ) plus the sign bit. Therefore, to represent this, at least 2 integer bits are needed. Similarly, the input image, being in greyscale, only requires values between 0 and 1. Again, here to be safe a range of  $[-2, 1.99]$  was applied. So, 2 integer bits were assigned for the image input variable type.

For the output of the convolutional layer, the number of bits changes. For reference, I included both the output before and after the Relu activation for future testing. As Relu sets any value of  $x < 0$  to equal 0, the only difference between the two variable types is the lower bound. However, since the magnitude of the lower bound is significantly less than the upper bound ( $0.29 \ll 4.3299$ ), both variables will have the same number of integer bits and variable type. To map 4.3299, 3 integer bits ( $2^3 = 8$ ) plus the integer bit are needed. So 4 bits are needed.

These assignments results in the following variable declarations in HLS:

```
typedef ap_fixed<16,2> pixel_t;    // input pixels
typedef ap_fixed<16,2> weight_t;   // kernel weights / bias
typedef ap_fixed<16,4> out_t;      // conv output after ReLU
typedef ap_fixed<24,8> acc_t;      // accumulator
```

It is important to note that the accumulation variable `acc_t` used during the convolution operation was given a larger amount of total bits and integer bits as well. This is to help with precision during the addition operations.

### 3) HLS Implementation of `Im2Col()` and `GemmConv2d0()` functions:

The next task is to implement the two functions `Im2col()` and `GemmConv2d0()` into cpp. The first step is to implement the `Im2col()` function following the same logic as in the python script:

```
def Im2Col(a, x_nrows, x_ncols, filter_nrows, filter_ncols,
          im2col_ofm_nrows, im2col_ofm_ncols, filter_size):
    # allocate the Im2Col() output matrix
    z = np.zeros((filter_size, im2col_ofm_nrows*im2col_ofm_ncols))

    col = 0

    for m in range(0, (x_nrows-filter_nrows+1)):
        for n in range(0, (x_ncols-filter_ncols+1)):
            row_offset = 0
            k = 0
```

```

for p in range(0, filter_ncols):
    for q in range(0, filter_nrows):
        z[row_offset + k, col] = a[m+p, n+q]
        k = k + 1
    col = col + 1
return z

```

The point of this script is to turn the convolution operation (essentially the same as correlation) into a matrix multiplication. So given a weight matrix  $w$  and an input image matrix  $x$

$$\mathbf{W} = \begin{bmatrix} w_{0,0} & w_{0,1} \\ w_{1,0} & w_{1,1} \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} & x_{0,3} \\ x_{1,0} & x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,0} & x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,0} & x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix}$$

We can turn the image matrix into

$$\hat{\mathbf{X}} = \begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} & x_{1,0} & x_{1,1} & x_{1,2} & x_{2,0} & x_{2,1} & x_{2,2} \\ x_{0,1} & x_{0,2} & x_{0,3} & x_{1,1} & x_{1,2} & x_{1,3} & x_{2,1} & x_{2,2} & x_{2,3} \\ x_{1,0} & x_{1,1} & x_{1,2} & x_{2,0} & x_{2,1} & x_{2,2} & x_{3,0} & x_{3,1} & x_{3,2} \\ x_{1,1} & x_{1,2} & x_{1,3} & x_{2,1} & x_{2,2} & x_{2,3} & x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix}$$

And subsequently do the following matrix multiplication instead

$$\underbrace{\begin{bmatrix} y_{0,0} & y_{0,1} & y_{1,0} & y_{1,1} \end{bmatrix}}_{\hat{\mathbf{Y}}} = \underbrace{\begin{bmatrix} w_{0,0} & w_{0,1} & w_{1,0} & w_{1,1} \end{bmatrix}}_{\hat{\mathbf{W}}} \times \underbrace{\begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} & x_{0,3} & x_{0,4} & x_{0,5} & x_{0,6} & x_{0,7} & x_{0,8} \\ x_{1,0} & x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} & x_{1,5} & x_{1,6} & x_{1,7} & x_{1,8} \\ x_{2,0} & x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} & x_{2,5} & x_{2,6} & x_{2,7} & x_{2,8} \\ x_{3,0} & x_{3,1} & x_{3,2} & x_{3,3} & x_{3,4} & x_{3,5} & x_{3,6} & x_{3,7} & x_{3,8} \end{bmatrix}}_{\hat{\mathbf{X}}}$$

So in order to do this matrix multiplication the `im2col` function needs to turn the the existing 28\*28 matrix into a 9 x 26\*26 matrix. This number is calculated through the logic described in the jupyter notebook:

```

top_pad = 0
left_pad = 0
x_nrows = 28
x_ncols = 28
filter_nrows = 3
filter_ncols = 3
# ofm = Output Feature Map

```

```

im2col_ofm_nrows = (x_nrows - filter_nrows + 2*top_pad) + 1
im2col_ofm_ncols = (x_ncols - filter_ncols + 2*left_pad) + 1
filter_size = filter_nrows*filter_ncols

```

The resulting  $x_{\text{hat}}$  matrix size consists of:

# of rows = Filter\_size = 9,  
 and,  
 # of cols = m2col\_ofm\_nrows \* im2col\_ofm\_ncols = 26\*26 = 676

As mentioned previously, the number 26\*26 makes sense as we do not include any padding in the convolution operation. Therefore, for this implementation this means that the resulting GEMM matrix is of size 9 x 26\*26. To then do the full convolution operation we then must flatten the weight 3x3 matrix into a 1x9 matrix. Multiplying these two matrices together results in a 1x(26x26) which is then, following the notebook logic, reshaped into a matrix of size m2col\_ofm\_nrows x im2col\_ofm\_ncols = 26x26. This exactly matches the same output size listed in the original keras conv2d function in the model description listed at the beginning.

Copying this same logic into cpp results in:

```

void Im2col(
    const pixel_t x[IMG_H][IMG_W],
    pixel_t col[FILTER_SIZE][OFM_SIZE]
){
    #pragma HLS PIPELINE II=1
    #pragma HLS ARRAY_PARTITION variable=col complete dim=1
    int out_col = 0;

    for (int r = 0; r < OUT_H; r++) {
        for (int c = 0; c < OUT_W; c++) {

            int idx = 0;

            for (int kr = 0; kr < K_H; kr++) {
                #pragma HLS UNROLL
                for (int kc = 0; kc < K_W; kc++) {
                    #pragma HLS UNROLL
                    col[idx][out_col] = x[r + kr][c + kc];
                    idx++;
                }
            }
            out_col++;
        }
    }
}

```

Where the col is the  $x_{\text{hat}}$  or GEMM version of x. We can see that x is defined as a matrix of size filter\_size and OFM\_size which are declared in the header file as:

```
#define FILTER_SIZE (K_H * K_W) // 9
#define OFM_SIZE (OUT_H * OUT_W) // 676
```

This output, col[9][676], matches the same logic and size as described previously.

The next step is to now optimize this function. I applied both a pipeline of  $II = 1$  and unrolled in this function. Firstly the unrolls are applied to effectively produce 1 im2col column every iteration. This means that the two inner most loops are iterating over the 3x3 kernel. By unrolling both this does 9 cycles per patch, causing the loop to disappear. In one cycle the following operation is done:

```
col[0][out_col] = x[r+0][c+0]
col[1][out_col] = x[r+0][c+1]
col[2][out_col] = x[r+0][c+2]
col[3][out_col] = x[r+1][c+0]
col[4][out_col] = x[r+1][c+1]
col[5][out_col] = x[r+1][c+2]
col[6][out_col] = x[r+2][c+0]
col[7][out_col] = x[r+2][c+1]
col[8][out_col] = x[r+2][c+2]
```

This means that we should expect a  $II = \text{number of columns} = 26 \times 26 = 676$ . I also partitioned the col array in order to supply enough reads for the unrolling. Furthermore, I also applied the pipeline  $II = 1$  directive to improve the overall performance.

Once the intermediate GEMM matrix is completed the next task is to compute the matrix multiplication between the flattened kernel and the GEMM matrix. This is computed by the following function:

```
void GemmConv2d0 (
    const pixel_t col[FILTER_SIZE][OFM_SIZE],
    out_t y[OUT_H][OUT_W]
) {
    #pragma HLS ARRAY_PARTITION variable=conv2d_0_w complete dim=1
    #pragma HLS ARRAY_PARTITION variable=col complete dim=1

    for (int c = 0; c < OFM_SIZE; c++) {
        #pragma HLS PIPELINE II=1
        acc_t acc = 0;
        for (int k = 0; k < FILTER_SIZE; k++) {
            #pragma HLS UNROLL
            acc += (acc_t)conv2d_0_w[k] * (acc_t)col[k][c];
        }

        acc += (acc_t)conv2d_0_b;
    }
}
```

```

        // ReLU
        if (acc < 0)
            acc = 0;

        int r = c / OUT_W;
        int cc = c % OUT_W;
        y[r][cc] = (out_t)acc;
    }
}

```

This function takes in the col output from the im2col function and uses nested loops to compute a matrix multiplication exactly like in the previous lab. Here after each (row,col) matrix point is calculated by multiplying the flattened kernel matrix row by the column of the col matrix, the bias term is then added. Subsequently, the ReLU function is applied. As mentioned previously, the ReLU function is 0 for when the input is < 0. Else it is just equal to the input. The resulting two lines of the function sneakily reformat the value into the correct y matrix position, which is of size 26x26.

For this function, I also partitioned the weight array and col (output of prev function) to make the pipelining and unrolling more efficient in the same way as was done for im2col. The unrolling on the inner loop, unrolls the columns of the im2col matrix, allowing for 9 parallel calculations.

Finally, both of these functions are called through the top-level function called Conv2D

```

void Conv2D(
    const pixel_t x[IMG_H][IMG_W],
    out_t y[OUT_H][OUT_W]
){
    #pragma HLS DATAFLOW

    pixel_t col[FILTER_SIZE][OFM_SIZE];

    Im2col(x, col);
    GemmConv2d0(col, y);
}

```

Through my testing, I quickly realized that getting either Im2col or GemmConv2d0 to be  $\geq$  to the desired  $II = 26 \times 26 = 676$  was the easy part. Getting the top level function to have the same  $II$  was difficult as the total  $II$  was equal to the Im2col interval + Gemm interval. This is where the dataflow pragma helped. Doing some research online revealed that this pragma, causes HLS to try to overlap the producer and consumer tasks, so the total interval becomes closer to  $\max(\text{Im2col interval}, \text{Gemm interval})$ . This means that both of the functions just have to have a  $II \leq 676$  to meet the required specs.

The results of the listed pragma's in c synthesis are shown below:

| MODULES & LOOPS     | LATENCY(NS) | ITERATION LATENCY | INTERVAL | TRIP COUNT | PIPELINED     | STRUCTURE | BRAM | DSP | FF  | LUT    | URAM |
|---------------------|-------------|-------------------|----------|------------|---------------|-----------|------|-----|-----|--------|------|
| ▼ Conv2D (2)        | 1.081E4     |                   | 676      |            | dataflow      | function  | 18   | 9   | 983 | 39,082 | 0    |
| ● Im2col            | 3.920E3     |                   | 392      |            | no            | function  | 0    | 0   | 394 | 38,746 | 0    |
| > ● GemmConv2d0 (1) | 6.880E3     |                   | 676      |            | loop auto-rew | function  | 0    | 9   | 588 | 321    | 0    |

We can see how using the dataflow pragma allows for the GemmConv2d0 function to have an II = 676 and Im2col = 392, yet overall cause the Conv2D function to have a II = 676. Furthermore, this implementation uses only 9 DSPs yet uses a significant amount of LUTs with 39,082. The significant majority of these LUTs come from the Im2Col function which uses 0 DSPs.

As we can see from the c synthesis the GemmConv3d0 function is the limiting factor in terms of II. Therefore in order to improve this, I tried to move the pipeline II = 1 pragma from the outer loop:

```
void GemmConv2d0(
    const pixel_t col[FILTER_SIZE][OFM_SIZE],
    out_t y[OUT_H][OUT_W]
){
    #pragma HLS ARRAY_PARTITION variable=conv2d_0_w complete dim=1
    #pragma HLS ARRAY_PARTITION variable=col complete dim=1

    for (int c = 0; c < OFM_SIZE; c++) {
        #pragma HLS PIPELINE II=1
```

To now being un-nested:

```
void GemmConv2d0(
    const pixel_t col[FILTER_SIZE][OFM_SIZE],
    out_t y[OUT_H][OUT_W]
){
    #pragma HLS ARRAY_PARTITION variable=conv2d_0_w complete dim=1
    #pragma HLS ARRAY_PARTITION variable=col complete dim=1
    #pragma HLS PIPELINE II=1

    for (int c = 0; c < OFM_SIZE; c++) {
```

This new placement cause c-synthesis to now run for over 20 minutes before producing a result. Below we can see that now this reduced the II from 676 to 348. Now the im2col function is the limiting factor in terms of overall II. At the cost of this efficiency however, the GemmConv2d0 function now uses an exorbitant amount of hardware. It now uses 6,084 DSPs and 77,135 LUTs. I do not know if decreasing the II by half is worth the significant increase in hardware utilization.



```

mae /= OFM_SIZE;

std::cout << "\n===== \n";
std::cout << "Conv2d0 verification results\n";
std::cout << "Max abs error   : " << std::fixed << std::setprecision(6) << max_abs_err <<
"\n";
std::cout << "Mean abs error   : " << std::fixed << std::setprecision(6) << mae << "\n";
std::cout << "Mismatch count   : " << mismatch_count << " / " << OFM_SIZE
<< "   (tol = " << tol << ") \n";
std::cout << "===== \n";

std::cout << "\nFirst 10 outputs (HW vs SW):\n";
for (int i = 0; i < 10; i++) {
    int r = i / OUT_W;
    int c = i % OUT_W;
    std::cout << "   [" << i << "] "
<< (float)y[r][c] << "   vs   "
<< gold_relu[i] << "\n";
}

return 0;
}

```

This testbench results in the following results:

```

=====
Conv2d0 verification results
Max abs error   : 0.000728
Mean abs error   : 0.000153
Mismatch count   : 0 / 676   (tol = 0.050000)
=====

First 10 outputs (HW vs SW):
[0] 0.000000   vs   0.000000
[1] 0.000000   vs   0.000000
[2] 0.000000   vs   0.000000
[3] 0.000000   vs   0.000000
[4] 0.000000   vs   0.000000
[5] 0.000000   vs   0.000000
[6] 0.000000   vs   0.000000
[7] 0.000000   vs   0.000000
[8] 0.000000   vs   0.000000
[9] 0.000000   vs   0.000000

```

With the tolerance of 0.05, we can see that GEMM implementation is completely accurate. The max error is only 0.0007, well below the tolerance. Furthermore, an average error of 0.0001 demonstrates that the logic of my functions is correct. I believe that this error comes down to the difference in precision when using floats versus a fixed number of integer bits. The testbench also shows 10 outputs compared to the .dat file. We can see that the first ten outputs were negative and thus Relu clamped them to = 0.